

Variant Calling

Michael Schatz

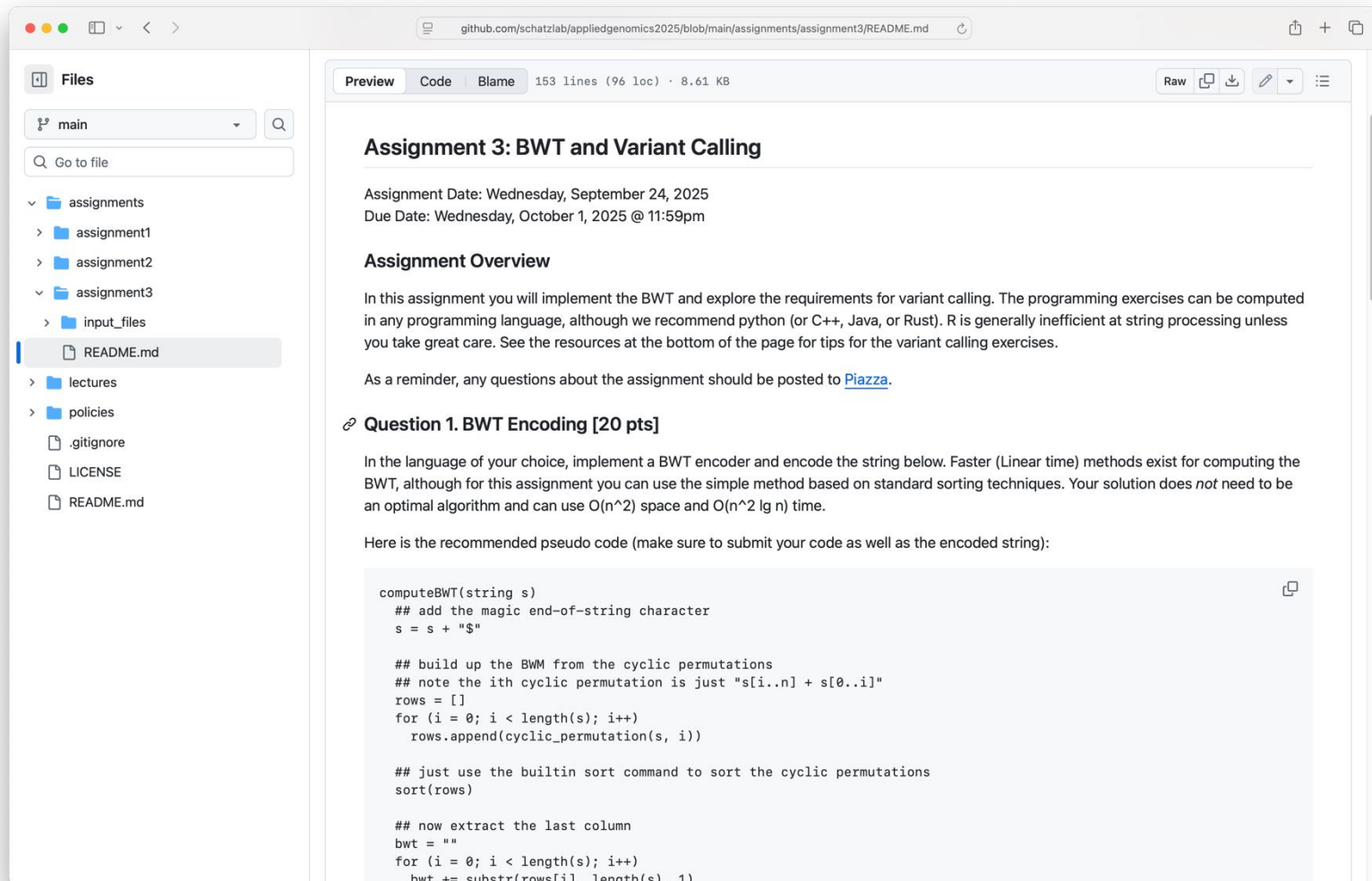
Sept 24, 2025

Lecture 9. Applied Comparative Genomics



Assignment 3: Variant Calling

Due Wed Oct 1 by 11:59pm



The screenshot shows a GitHub repository page for 'schatzlab/appliedgenomics2025'. The left sidebar displays the file structure, with 'assignment3/README.md' selected. The main content area shows the 'README.md' file, which includes the assignment title, dates, overview, and a programming exercise.

Assignment 3: BWT and Variant Calling

Assignment Date: Wednesday, September 24, 2025
Due Date: Wednesday, October 1, 2025 @ 11:59pm

Assignment Overview

In this assignment you will implement the BWT and explore the requirements for variant calling. The programming exercises can be computed in any programming language, although we recommend python (or C++, Java, or Rust). R is generally inefficient at string processing unless you take great care. See the resources at the bottom of the page for tips for the variant calling exercises.

As a reminder, any questions about the assignment should be posted to [Piazza](#).

Question 1. BWT Encoding [20 pts]

In the language of your choice, implement a BWT encoder and encode the string below. Faster (Linear time) methods exist for computing the BWT, although for this assignment you can use the simple method based on standard sorting techniques. Your solution does *not* need to be an optimal algorithm and can use $O(n^2)$ space and $O(n^2 \lg n)$ time.

Here is the recommended pseudo code (make sure to submit your code as well as the encoded string):

```
computeBWT(string s)
  ## add the magic end-of-string character
  s = s + "$"

  ## build up the BWM from the cyclic permutations
  ## note the ith cyclic permutation is just "s[i..n] + s[0..i]"
  rows = []
  for (i = 0; i < length(s); i++)
    rows.append(cyclic_permutation(s, i))

  ## just use the builtin sort command to sort the cyclic permutations
  sort(rows)

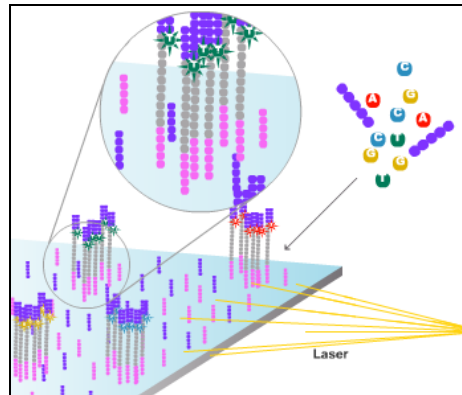
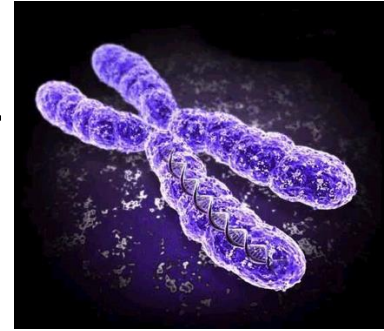
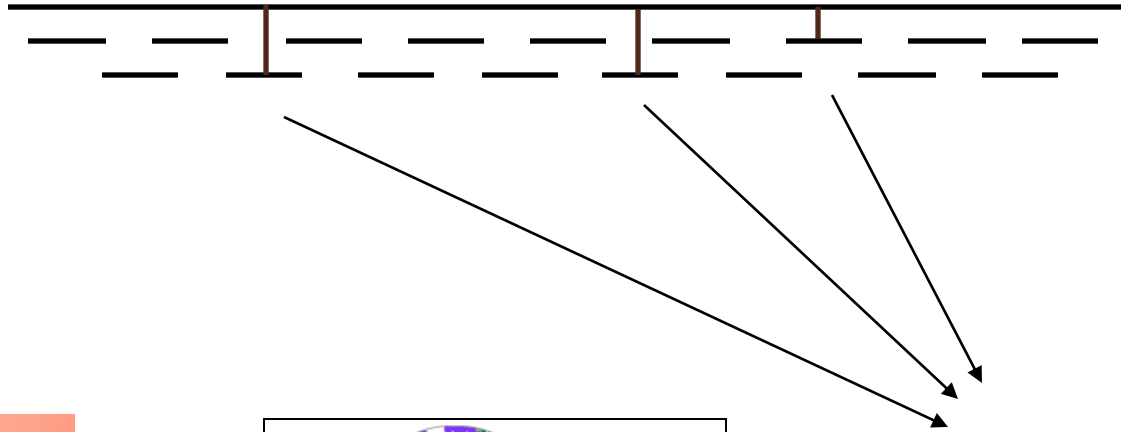
  ## now extract the last column
  bwt = ""
  for (i = 0; i < length(s); i++)
    bwt += substr(rows[i], length(s), 1)
```

<https://github.com/schatzlab/appliedgenomics2025/tree/main/assignments/assignment3>

Check Piazza for questions!

Personal Genomics

How does your genome compare to the reference?



Heart Disease

Cancer

Amazing actress

Algorithm Overview

1. Split read into segments

Read

CCAGTAGCTCTCAGCCTTATTTTACCCAGGCCTGTA

Read (reverse complement)

TACAGGCCTGGGTAAAATAAGGCTGAGAGCTACTGG

Policy: extract 16 nt seed every 10 nt

Seeds

+, 0: CCAGTAGCTCTCAGCC

+, 10: TCAGCCTTATTTACC

+, 20: **TTACCCAGGCCTGTA**

-, 0: **TACAGGCCTGGGTAAA**

-, 10: GGTAAAATAAGGCTGA

-, 20: GGCTGAGAGCTACTGG

2. Lookup each segment and prioritize

Seeds

+, 0: CCAGTAGCTCTCAGCC

+, 10: **TCAGCCTTATTTACC**

+ , 20: TTTACCCAGGCCTGTA

-, 0: **TACAGGCCTGGGTAAA**

-, 10: **GGTAAAATAAGGCTGA**

-, 20: GGCTGAGAGCTACTGG

Ungapped
alignment with
FM Index

Diagram illustrating a sequence alignment between a reference sequence and a query sequence. The reference sequence is "a a c" and the query sequence is "\$ a c a a c g". The alignment shows the query sequence being mapped to the reference sequence, with a blue "I" indicating an insertion at the start of the query sequence.

Seed alignments (as B ranges)

$$\{ [211, 212], [212, 214] \}$$
$$\{ [653, 654], [651, 653] \}$$

{ [684, 685] }

{ }

{ }

{ [624, 625] }

3. Evaluate end-to-end match

Extension candidates

SA:684, chr12:1955

SA:624, chr2:462

SA:213: chr12:1935

SA:652: chr12:1945

SIMD dynamic
programming
aligner

SAM alignments

```

r1      0      chr12      1936      0
      36M      *      0      0
      CCAGTAGCTCTCAGCCTTATTTTACCCAGGCCTGTA
      IIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIII
      AS:i:0      XS:i:-2      XN:i:0
      XM:i:0      XO:i:0      XG:i:0
      NM:i:0      MD:Z:36      YT:Z:UU
      YM:i:0

```

(Langmead & Salzberg, 2012)

Exact Matching Review & Overview

Where is GATTACA in the human genome?

Brute Force
(3 GB)

BANANA
BAN
ANA
NAN
ANA

$O(m * n)$

Slow & Easy

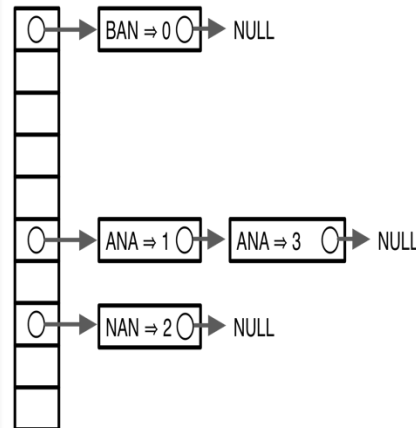
Suffix Array
(>15 GB)

6	\$
5	A\$
3	ANA\$
1	ANANA\$
0	BANANA\$
4	NA\$
2	NANA\$

$O(m + \lg n)$

Full-text index

Hash Table
(>15 GB)



$O(1)$

Fixed-length lookup

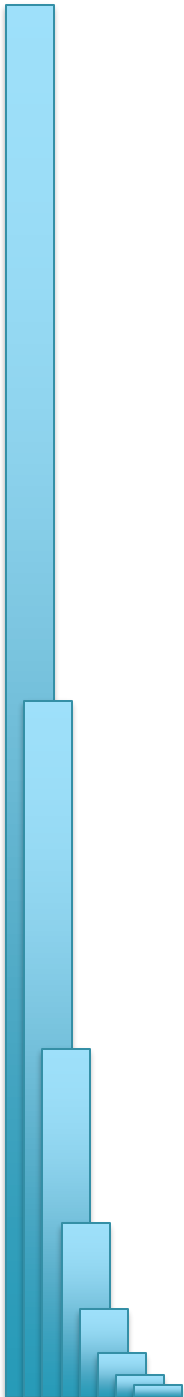
BWT
(3 GB)

BANANA\$
=>
\$BANANA**A**
A\$BANAN**N**
ANASBAN**N**
ANANA\$**B**
BANANA\$**S**
NASBAN**A**
NANA\$BA**A**
=>
ANNB\$AA

$O(m)$

Full-text and concise

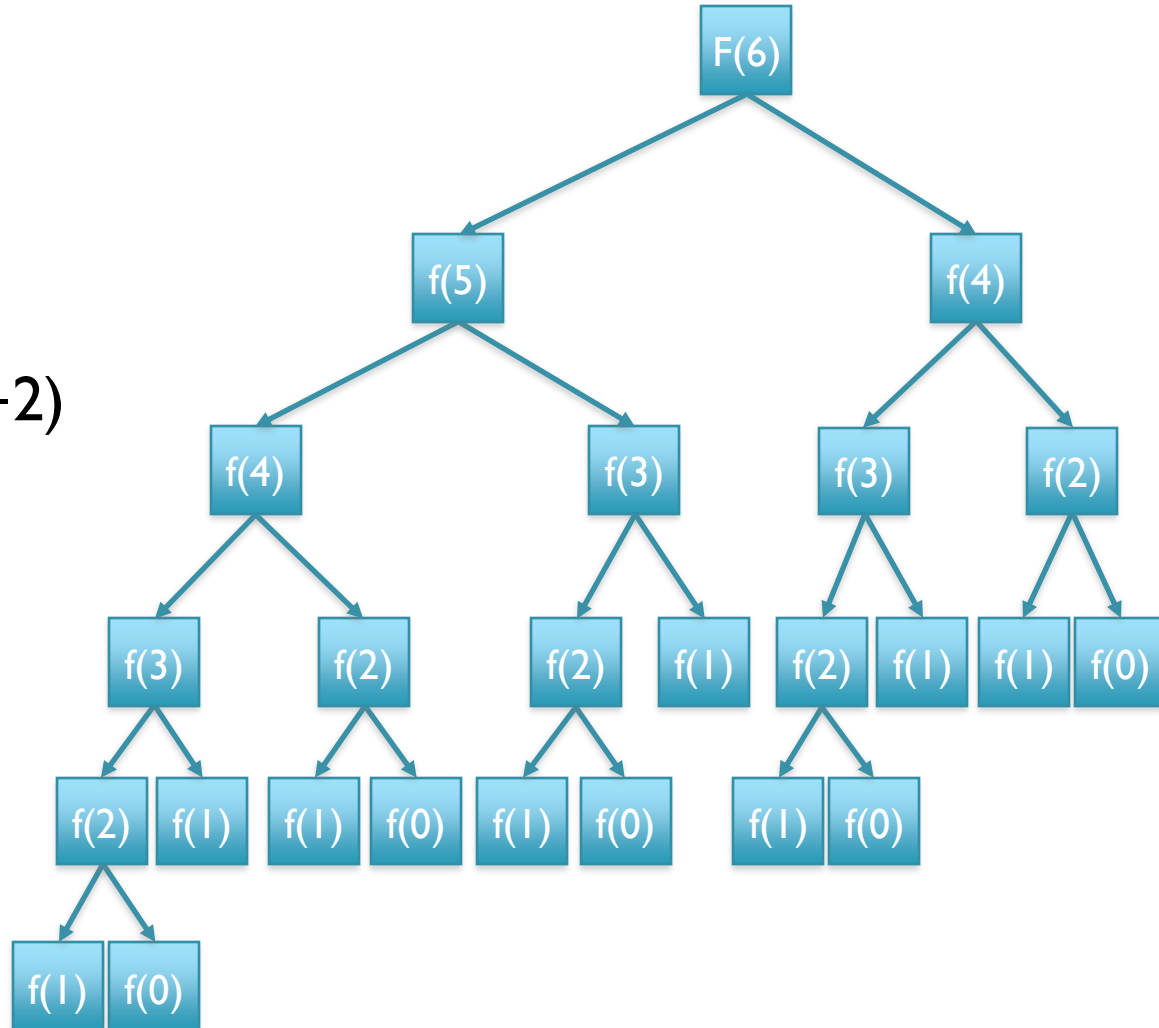
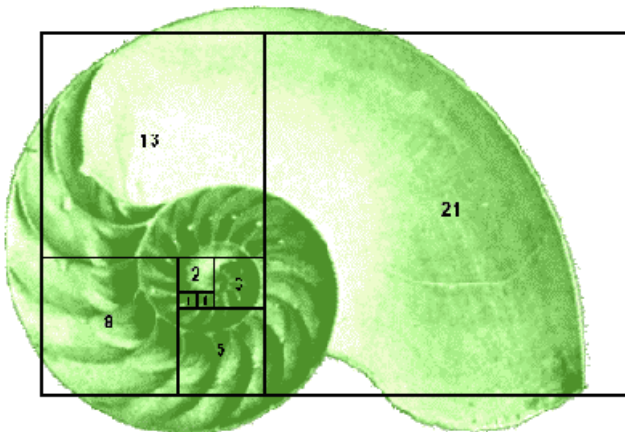
*** These are general techniques applicable to any text search problem ***



Dynamic Programming

Fibonacci Sequence

```
def fib(n):  
    if n == 0 or n == 1:  
        return n  
    else:  
        return fib(n-1) + fib(n-2)
```

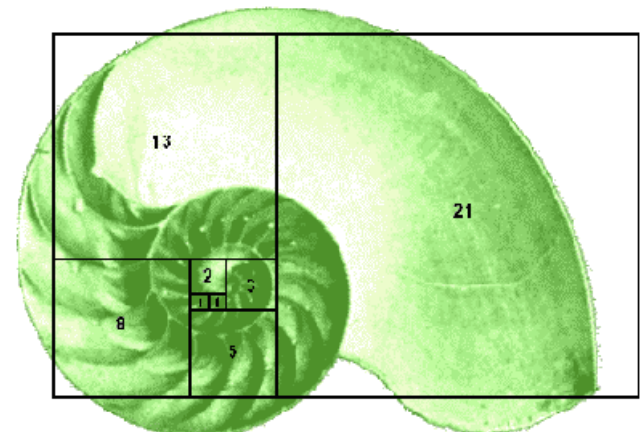


Bottom-up Fibonacci Sequence

```
def fib(n):  
    table = [0] * (n+1)  
    table[0] = 0  
    table[1] = 1  
    for i in range(2,n+1):  
        table[i] = table[i-2] + table[i-1]  
    return table[n]
```

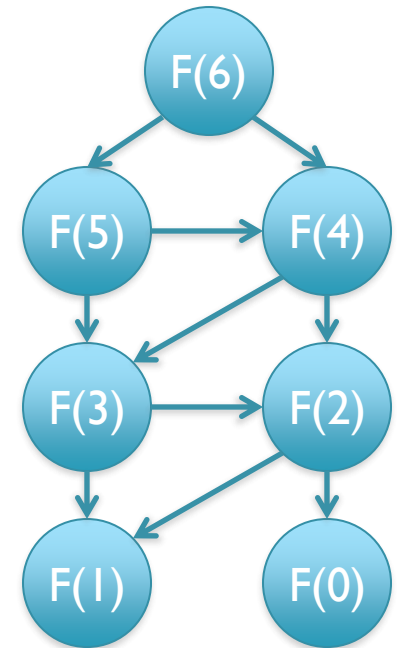
0	1	2	3	4	5	6
0	1	1	2	3	5	8

[What is the running time?]



Dynamic Programming

- General approach for solving (some) complex problems
 - When applicable, the method takes far less time than naive methods.
 - Polynomial time ($O(n)$ or $O(n^2)$) instead of exponential time ($O(2^n)$ or $O(3^n)$)
- Requirements:
 - **Overlapping subproblems**
 - **Optimal substructure**
- Applications:
 - Fibonacci
 - Longest Increasing Subsequence (Bonus Slides!)
 - Sequence alignment, Dynamic Time Warp, Viterbi
- Not applicable:
 - Traveling salesman problem, Clique finding, Subgraph isomorphism, ...
 - The cheapest flight from airport A to airport B involves a single connection through airport C, but the cheapest flight from airport A to airport C involves a connection through some other airport D.



Similarity metrics

- Hamming distance

- Count the number of substitutions to transform one string into another

MIKESCHATZ

| | X | | XXXX |

MICESHATZZ

5

- Edit distance

- The minimum number of substitutions, insertions, or deletions to transform one string into another

MIKESCHAT-Z

| | X | | X | | | X |

MICES-HATZZ

3

Edit Distance Example

AGCACACA → ACACACTA in 3 steps

AGCACACA → (1. change G to C)

ACCACACA → (2. delete C)

ACACACA → (3. insert T after 3rd C)

ACACACTA → done

[Is this the best we can do?]

Reverse Engineering Edit Distance

$$D(\text{AGCACACA}, \text{ACACACTA}) = ?$$

Imagine we already have the optimal alignment of the strings, the last column can only be 1 of 3 options:

... M	... I	... D
...A	...-	...A
...A	...A	...-

The optimal alignment of last two columns is then 1 of 9 possibilities

... MM	... IM	... DM	... MI	... II	... DI	... MD	... ID	... DD
...CA	...-A	...CA	...A-	...--	...A-	...CA	...-A	...CA
...TA	...TA	...-A	...TA	...TA	...-A	...A-	...A-	...--

The optimal alignment of the last three columns is then 1 of 27 possibilities...

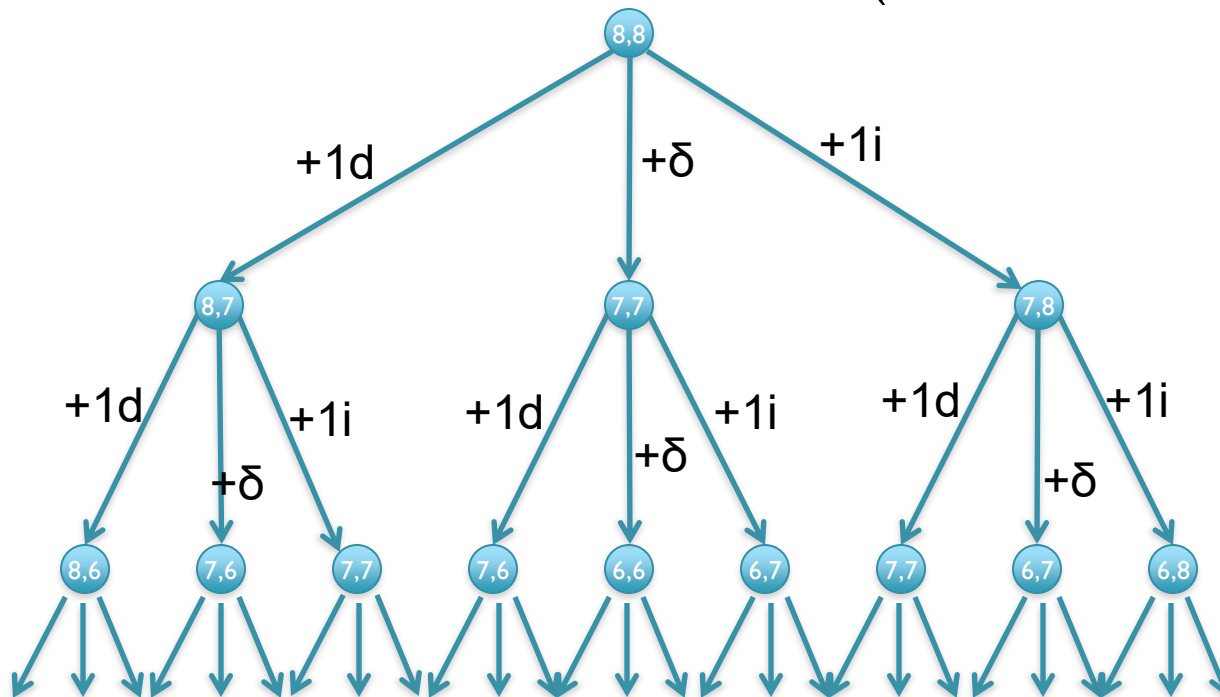
... M ...	... I ...	... D ...
...X...	...-...	...X...
...Y...	...Y...	...-...

Eventually spell out every possible sequence of {I,M,D}

Recursive solution

- Computation of D is a recursive process.
 - At each step, we only allow matches, substitutions, and indels
 - $D(i,j)$ in terms of $D(i', j')$ for $i' \leq i$ and $j' \leq j$.

$$D(\text{AGCACACA}, \text{ACACACTA}) = \min\{D(\text{AGCACACA}, \text{ACACACT}) + 1, \\ D(\text{AGCACAC}, \text{ACACACTA}) + 1, \\ D(\text{AGCACAC}, \text{ACACACT}) + \delta(\text{A}, \text{A})\}$$



[What is the running time?]

Dynamic Programming

- We could code this as a recursive function call...
...with an exponential number of function evaluations
- There are only $(n+1) \times (m+1)$ pairs i and j
 - We are evaluating $D(i,j)$ multiple times
- Compute $D(i,j)$ bottom up.
 - Start with smallest $(i,j) = (1,1)$.
 - Store the intermediate results in a table.
 - Compute $D(i,j)$ *after* $D(i-1,j)$, $D(i,j-1)$, and $D(i-1,j-1)$

Recurrence Relation for D

Find the edit distance (minimum number of operations to convert one string into another) in $O(mn)$ time

- Base conditions:

- $D(i,0) = i$, for all $i = 0, \dots, n$
- $D(0,j) = j$, for all $j = 0, \dots, m$

- For $i > 0, j > 0$:

$$D(i,j) = \min \left\{ \begin{array}{ll} D(i-1,j) + 1, & // \text{align } 0 \text{ chars from } S, 1 \text{ from } T \\ D(i,j-1) + 1, & // \text{align } 1 \text{ chars from } S, 0 \text{ from } T \\ D(i-1,j-1) + \delta(S(i),T(j)) & // \text{align } 1+1 \text{ chars} \end{array} \right\}$$

[Why do we want the min?]

Dynamic Programming Matrix

		A	C	A	C	A	C	T	A
	0	1	2	3	4	5	6	7	8
A	1								
G	2								
C	3								
A	4								
C	5								
A	6								
C	7								
A	8								

[What does the initialization mean?]

Dynamic Programming Matrix

		A	C	A	C	A	C	T	A
	0	I	2	3	4	5	6	7	8
A	1	0							
G	2								
C	3								
A	4								
C	5								
A	6								
C	7								
A	8								

$$D[A,A] = \min\{D[A,]+1, D[,A]+1, D[,]+\delta(A,A)\}$$

Dynamic Programming Matrix

		A	C	A	C	A	C	T	A
	0	1	2	3	4	5	6	7	8
A	1	0	1						
G	2								
C	3								
A	4								
C	5								
A	6								
C	7								
A	8								

$$D[A,AC] = \min\{D[A,A]+1, D[,AC]+1, D[,A]+\delta(A,C)\}$$

Dynamic Programming Matrix

		A	C	A	C	A	C	T	A
	0	1	2	3	4	5	6	7	8
A	1	0	1	2					
G	2								
C	3								
A	4								
C	5								
A	6								
C	7								
A	8								

$$D[A,ACA] = \min\{D[A,AC]+1, D[,ACA]+1, D[,AC]+\delta(A,A)\}$$

Dynamic Programming Matrix

		A	C	A	C	A	C	T	A
	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	<u>5</u>	<u>6</u>	<u>7</u>	<u>8</u>
A	1	0	1	2	3	4	5	6	<u>7</u>
G	2								
C	3								
A	4								
C	5								
A	6								
C	7								
A	8								

$$D[A, ACACACTA] = 7$$

-----A

***** |

ACACACTA

[What about the other A?]

Dynamic Programming Matrix

		A	C	A	C	A	C	T	A
	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	<u>5</u>	<u>6</u>	<u>7</u>	<u>8</u>
A	1	0	1	2	3	<u>4</u>	5	6	7
G	2	1	1	2	3	4	<u>5</u>	<u>6</u>	<u>7</u>
C	3								
A	4								
C	5								
A	6								
C	7								
A	8								

$$D[AG,ACACACTA] = 7$$

----AG--

***** | ***

ACACACTA

Dynamic Programming Matrix

		A	C	A	C	A	C	T	A
	<u>0</u>	1	2	3	4	5	6	7	8
A	1	<u>0</u>	1	2	3	4	5	6	7
G	2	<u>1</u>	1	2	3	4	5	6	7
C	3	2	<u>1</u>	2	2	3	4	5	6
A	4	3	2	<u>1</u>	2	2	3	4	5
C	5	4	3	2	<u>1</u>	2	2	3	4
A	6	5	4	3	2	<u>1</u>	2	3	3
C	7	6	5	4	3	2	<u>1</u>	<u>2</u>	3
A	8	7	6	5	4	3	2	2	<u>2</u>

$$D[AGCACACA, ACACACTA] = 2$$

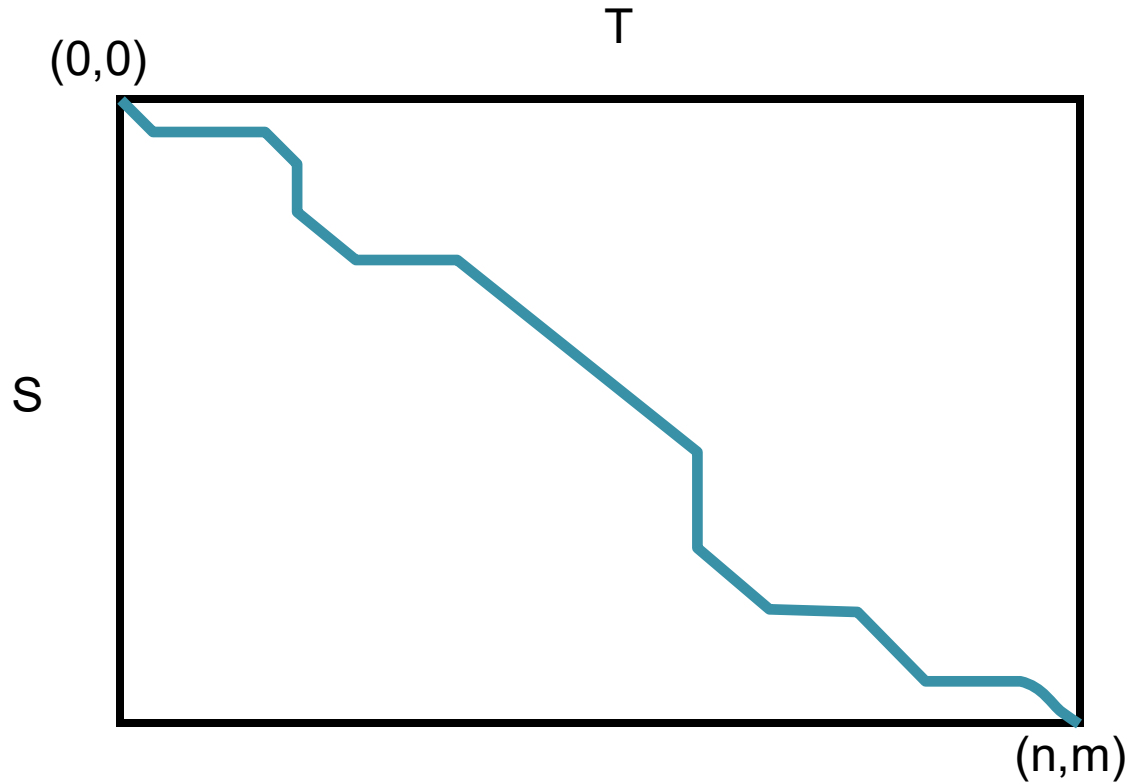
AGCACAC-A

| * | | | | * |

A-CACACTA

[Can we do it any better?]

Global Alignment Schematic

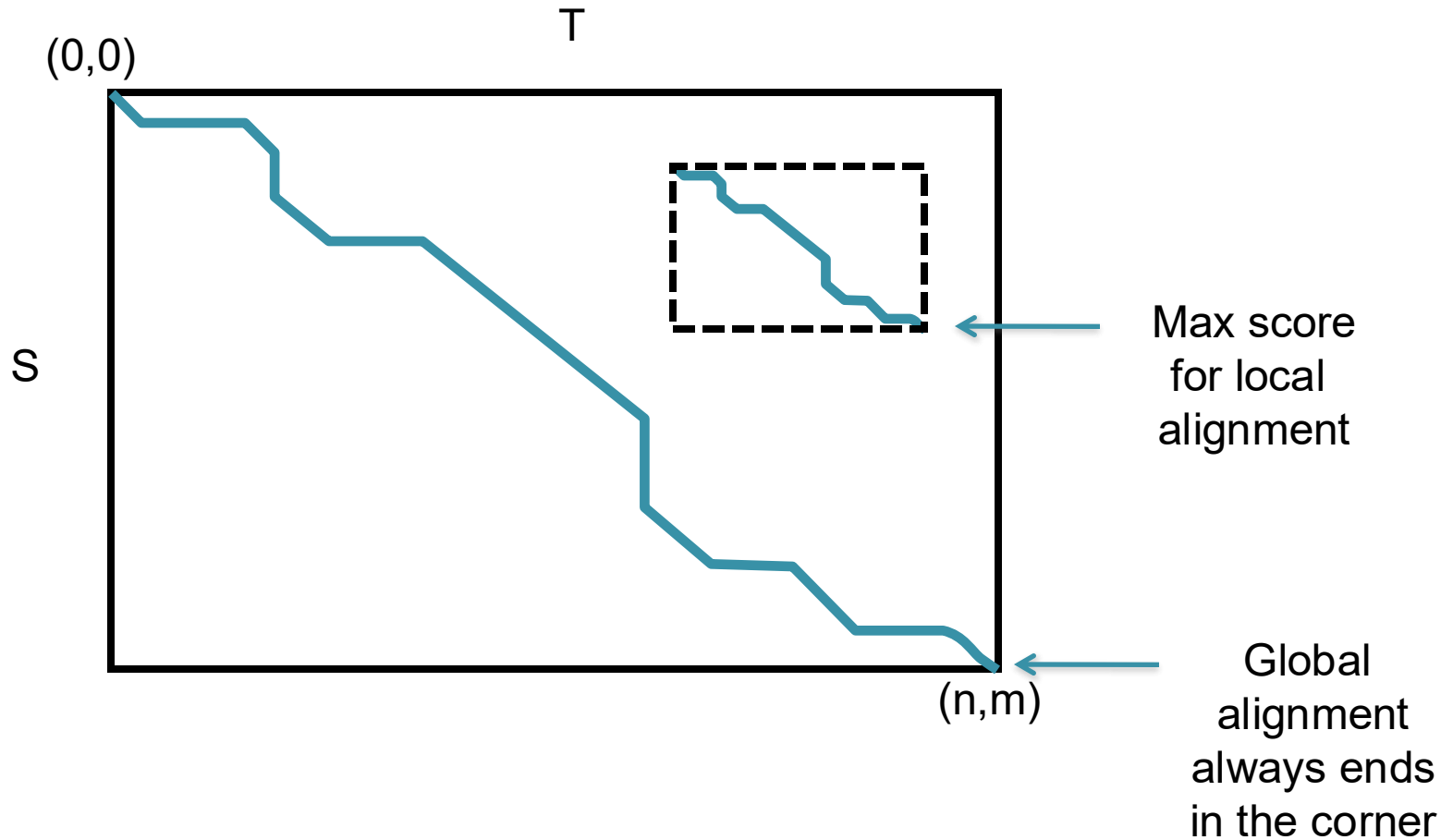


- A high quality alignment will stay close to the diagonal
 - If we are only interested in high quality alignments, we can skip filling in cells that can't possibly lead to a high quality alignment
 - Find the global alignment with at most edit distance d : $O(2dn)$

Local vs. Global Alignment

- The Global Alignment Problem tries to find the best end-to-end alignment between the two strings
 - Only applicable for very closely related sequences
- The Local Alignment Problem tries to find pairs of **substrings** with highest similarity.
 - Especially important if one string is substantially longer than the other
 - Especially important if there is only a distant evolutionary relationship

Global vs Local Alignment Schematic



Local vs. Global Alignment (cont' d)

- Global Alignment

```
--T--CC-C-AGT--TATGT-CAGGGGACACG-A-GCATGCAGA-GAC
|  ||| |  ||| |  ||| |||  ||| |  ||| ||| |  ||| |
AATTGCCGCC-GTCGT-T-TTCAG-----CA-GTTATG-T-CAGAT--C
```

- Local Alignment—better alignment to find conserved segment

```
          tccCAGTTATGTCAGgggacacgagcatgcagagac
            |||||
aattgccgccgtcgttttcagCAGTTATGTCAGatc
```

SAM / BAM Files

```
Coor      12345678901234  5678901234567890123456789012345
ref       AGCATGTTAGATAA**GATAGCTGTGCTAGTAGGCAGTCAGCGCCAT

+r001/1      TTAGATAAAGGATA*CTG
+r002        aaaAGATAA*GGATA
+r003        gcctaAGCTAA
+r004                ATAGCT.....TCAGC
-r003                ttagctTAGGC
-r001/2                        CAGCGGCAT
```

```
@HD VN:1.6 SO:coordinate
@SQ SN:ref LN:45
r001  99 ref  7 30 8M2I4M1D3M = 37 39 TTAGATAAAGGATACTG *
r002   0 ref  9 30 3S6M1P1I4M * 0 0 AAAAGATAAGGATA *
r003   0 ref  9 30 5S6M * 0 0 GCCTAAGCTAA * SA:Z:ref,29,-,6H5M,17,0;
r004   0 ref 16 30 6M14N5M * 0 0 ATAGCTTCAGC *
r003 2064 ref 29 17 6H5M * 0 0 TAGGC * SA:Z:ref,9,+,5S6M,30,1;
r001 147 ref 37 30 9M = 7 -39 CAGCGGCAT * NM:i:1
```

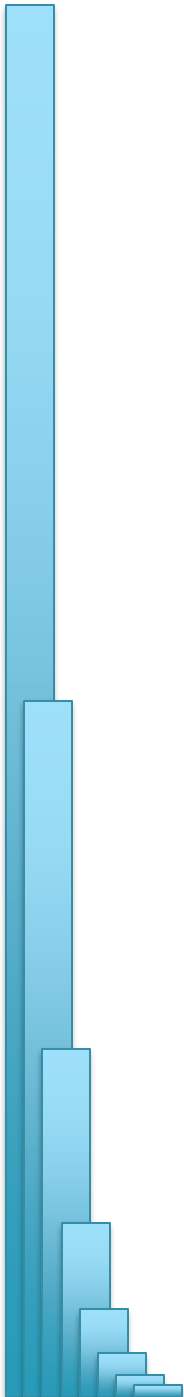
SAM / BAM Files

1.4 The alignment section: mandatory fields

In the SAM format, each alignment line typically represents the linear alignment of a segment. Each line consists of 11 or more TAB-separated fields. The first eleven fields are always present and in the order shown below; if the information represented by any of these fields is unavailable, that field's value will be a placeholder, either '0' or '*' as determined by the field's type. The following table gives an overview of these mandatory fields in the SAM format:

Col	Field	Type	Regex/Range	Brief description
1	QNAME	String	[!-?A-~]{1,254}	Query template NAME
2	FLAG	Int	[0, 2 ¹⁶ - 1]	bitwise FLAG
3	RNAME	String	* [:rname:^*=] [:rname:]*	Reference sequence NAME ¹¹
4	POS	Int	[0, 2 ³¹ - 1]	1-based leftmost mapping POSition
5	MAPQ	Int	[0, 2 ⁸ - 1]	MAPping Quality
6	CIGAR	String	* ([0-9]+[MIDNSHPX=])+	CIGAR string
7	RNEXT	String	* = [:rname:^*=] [:rname:]*	Reference name of the mate/next read
8	PNEXT	Int	[0, 2 ³¹ - 1]	Position of the mate/next read
9	TLEN	Int	[-2 ³¹ + 1, 2 ³¹ - 1]	observed Template LENgth
10	SEQ	String	* [A-Za-z=.]+	segment SEQUENCE
11	QUAL	String	[!-~]+	ASCII of Phred-scaled base QUALity+33

¹¹Reference sequence names may contain any printable ASCII characters with the exception of certain punctuation characters, and may not start with '*' or '='. See Section 1.2.1 for details and an explanation of the [:rname:] notation.

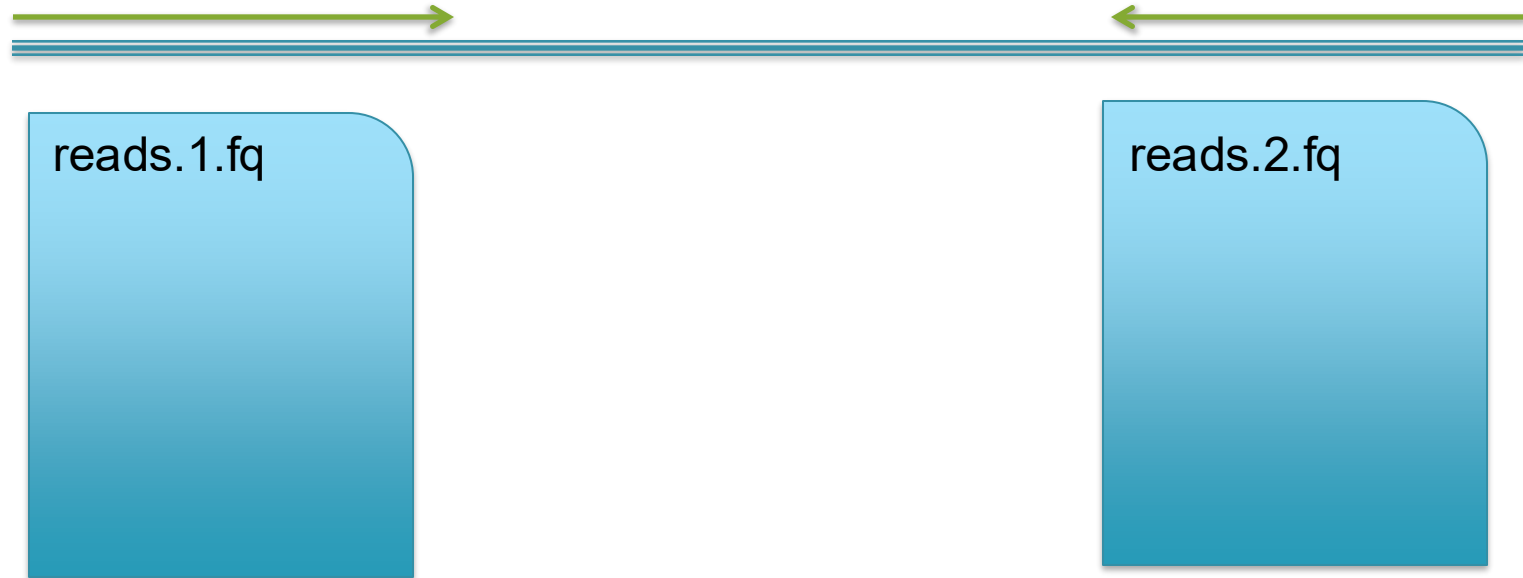


Part 2: Variant Calling

Variant Calling Overview



FASTQ Files



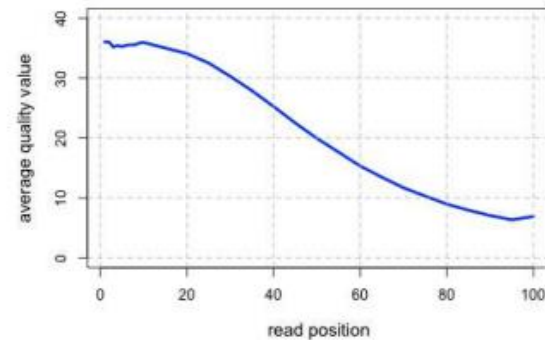
```
@SEQ_ID
GATTGTTGGGGTTCAAAGCAGTATCGATCAAATAGTAAATCCATTTGTTCAACTCACAGTTT
+
!''*(((((***+))%%#+))%%%).1***-+''')**55CCF>>>>>CCCCCCC65
```

@Identifier
Sequence
+Separator
Quality Values
...

Illumina Quality

QV	P _{error}
40	1/10000
30	1/1000
20	1/100
10	1/10

$$Q_{\text{sanger}} = -10 \log_{10} p$$



```

SSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSSS.....
.....XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX.....
.....IIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIII.....
.....JJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJ.....
LLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLL.....
!"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMN...
PQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~
|                               |               |   }
33                            59        64       73              104             126
```

- ```
S - Sanger Phred+33, raw reads typically (0, 40)
X - Solexa Solexa+64, raw reads typically (-5, 40)
I - Illumina 1.3+ Phred+64, raw reads typically (0, 40)
J - Illumina 1.5+ Phred+64, raw reads typically (3, 40)
 with 0=unused, 1=unused, 2=Read Segment Quality Control Indicator (bold)
 (Note: See discussion above).
L - Illumina 1.8+ Phred+33, raw reads typically (0, 41)
```

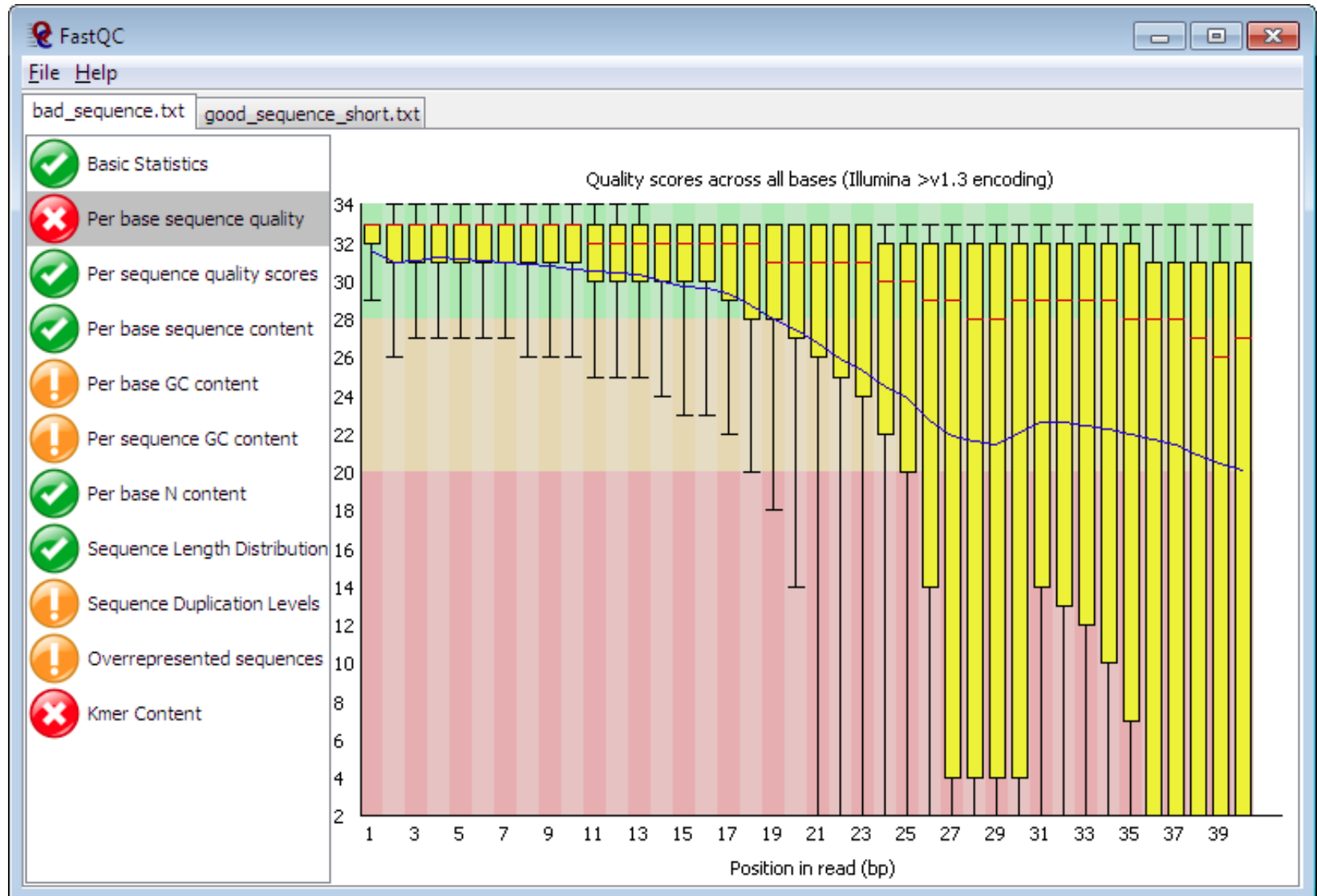


# ASCII Code

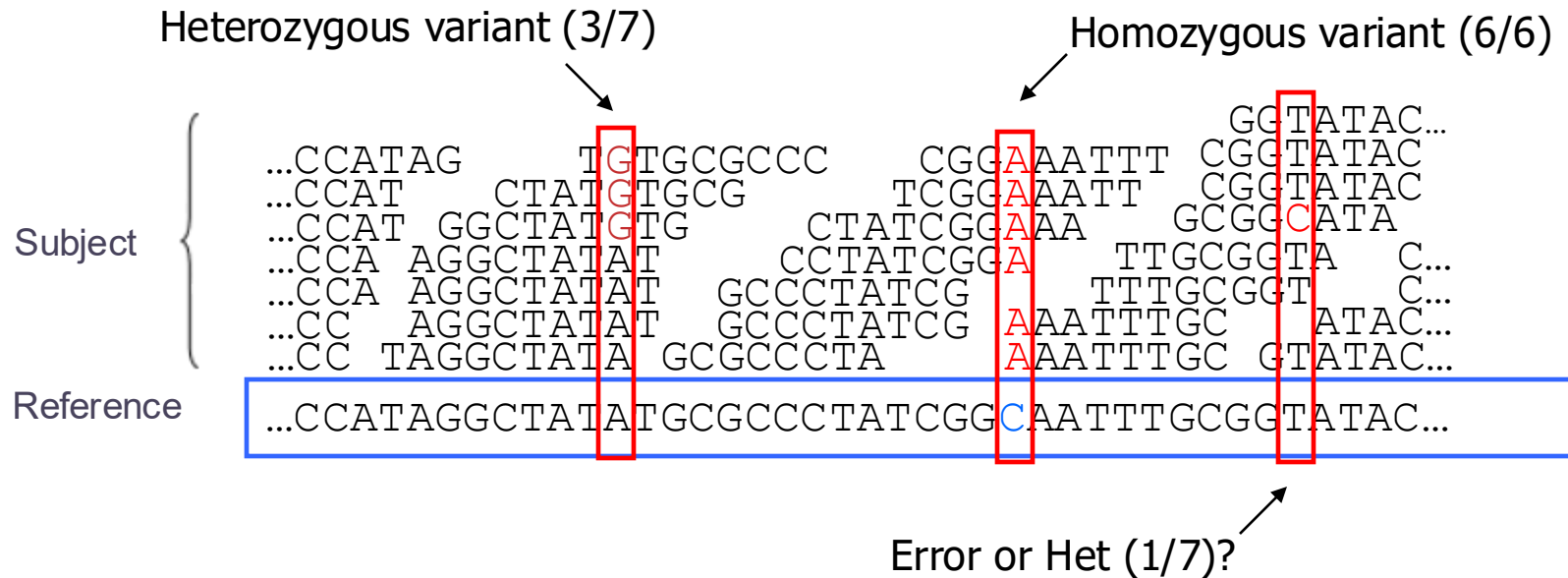
|    |     |    |     |    |    |    |   |    |   |    |   |     |   |     |     |     |     |     |   |     |   |     |   |     |   |     |   |     |   |     |   |
|----|-----|----|-----|----|----|----|---|----|---|----|---|-----|---|-----|-----|-----|-----|-----|---|-----|---|-----|---|-----|---|-----|---|-----|---|-----|---|
| 0  | NUL | 16 | DLE | 32 | SP | 48 | 0 | 64 | @ | 80 | P | 96  | ` | 112 | p   | 128 | €   | 144 |   | 160 |   | 176 | ° | 192 | À | 208 | Đ | 224 | à | 240 | đ |
| 1  | SOH | 17 | DC1 | 33 | !  | 49 | 1 | 65 | A | 81 | Q | 97  | a | 113 | q   | 129 |     | 145 | ' | 161 | ı | 177 | ± | 193 | Á | 209 | Ñ | 225 | á | 241 | ñ |
| 2  | STX | 18 | DC2 | 34 | "  | 50 | 2 | 66 | B | 82 | R | 98  | b | 114 | r   | 130 | ,   | 146 | ' | 162 | ¢ | 178 | ² | 194 | Â | 210 | Ò | 226 | â | 242 | ò |
| 3  | ETX | 19 | DC3 | 35 | #  | 51 | 3 | 67 | C | 83 | S | 99  | c | 115 | s   | 131 | f   | 147 | " | 163 | £ | 179 | ³ | 195 | Ã | 211 | Ó | 227 | ã | 243 | ó |
| 4  | EOT | 20 | DC4 | 36 | \$ | 52 | 4 | 68 | D | 84 | T | 100 | d | 116 | t   | 132 | "   | 148 | " | 164 | ¤ | 180 | ´ | 196 | Ä | 212 | Ô | 228 | ä | 244 | ô |
| 5  | ENQ | 21 | NAK | 37 | %  | 53 | 5 | 69 | E | 85 | U | 101 | e | 117 | u   | 133 | ... | 149 | • | 165 | ¥ | 181 | µ | 197 | Å | 213 | Õ | 229 | å | 245 | õ |
| 6  | ACK | 22 | SYN | 38 | &  | 54 | 6 | 70 | F | 86 | V | 102 | f | 118 | v   | 134 | †   | 150 | – | 166 | ı | 182 | ¶ | 198 | Æ | 214 | Ö | 230 | æ | 246 | ö |
| 7  | BEL | 23 | ETB | 39 | '  | 55 | 7 | 71 | G | 87 | W | 103 | g | 119 | w   | 135 | #   | 151 | — | 167 | § | 183 | · | 199 | Ç | 215 | × | 231 | ç | 247 | ÷ |
| 8  | BS  | 24 | CAN | 40 | (  | 56 | 8 | 72 | H | 88 | X | 104 | h | 120 | x   | 136 | ^   | 152 | ~ | 168 | ¨ | 184 | ¸ | 200 | È | 216 | Ø | 232 | è | 248 | ø |
| 9  | HT  | 25 | EM  | 41 | )  | 57 | 9 | 73 | I | 89 | Y | 105 | i | 121 | y   | 137 | ‰   | 153 | ™ | 169 | © | 185 | ¹ | 201 | É | 217 | Ù | 233 | é | 249 | ù |
| 10 | LF  | 26 | SUB | 42 | *  | 58 | : | 74 | J | 90 | Z | 106 | j | 122 | z   | 138 | Š   | 154 | š | 170 | ª | 186 | º | 202 | Ê | 218 | Ú | 234 | ê | 250 | ú |
| 11 | VT  | 27 | ESC | 43 | +  | 59 | ; | 75 | K | 91 | [ | 107 | k | 123 | {   | 139 | <   | 155 | > | 171 | « | 187 | » | 203 | Ë | 219 | Û | 235 | ë | 251 | û |
| 12 | FF  | 28 | FS  | 44 | ,  | 60 | < | 76 | L | 92 | \ | 108 | l | 124 |     | 140 | Œ   | 156 | œ | 172 | ¬ | 188 | ¼ | 204 | Ì | 220 | Ü | 236 | ì | 252 | ü |
| 13 | CR  | 29 | GS  | 45 | -  | 61 | = | 77 | M | 93 | ] | 109 | m | 125 | }   | 141 |     | 157 |   | 173 |   | 189 | ½ | 205 | Í | 221 | Ý | 237 | í | 253 | ý |
| 14 | SO  | 30 | RS  | 46 | .  | 62 | > | 78 | N | 94 | ^ | 110 | n | 126 | ~   | 142 | Ž   | 158 | ž | 174 | ® | 190 | ¾ | 206 | Î | 222 | Þ | 238 | î | 254 | þ |
| 15 | SI  | 31 | US  | 47 | /  | 63 | ? | 79 | O | 95 | _ | 111 | o | 127 | DEL | 143 |     | 159 | ÿ | 175 | - | 191 | ¿ | 207 | Ï | 223 | ß | 239 | ï | 255 | ÿ |

<https://en.wikipedia.org/wiki/ASCII>

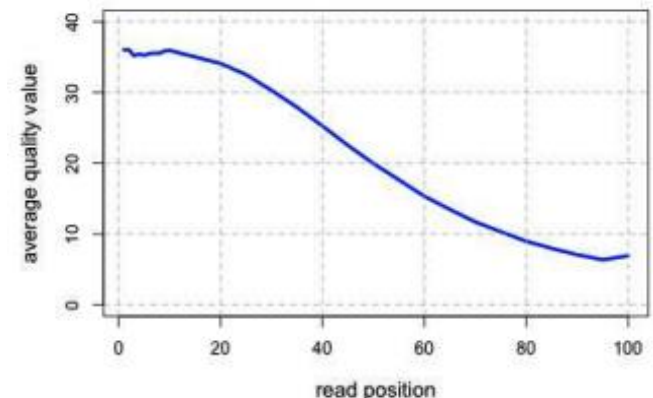
# FASTQC: Are my data any good?



# Genotyping Theory



- If there were no sequencing errors, identifying SNPs would be very easy: any time a read disagrees with the reference, it must be a variant!
- Sequencing instruments make mistakes
  - Quality of read decreases over the read length
- A single read differing from the reference is probably just an error, but it becomes more likely to be real as we see it multiple times



# The Binomial Distribution: Adventures in Coin Flipping

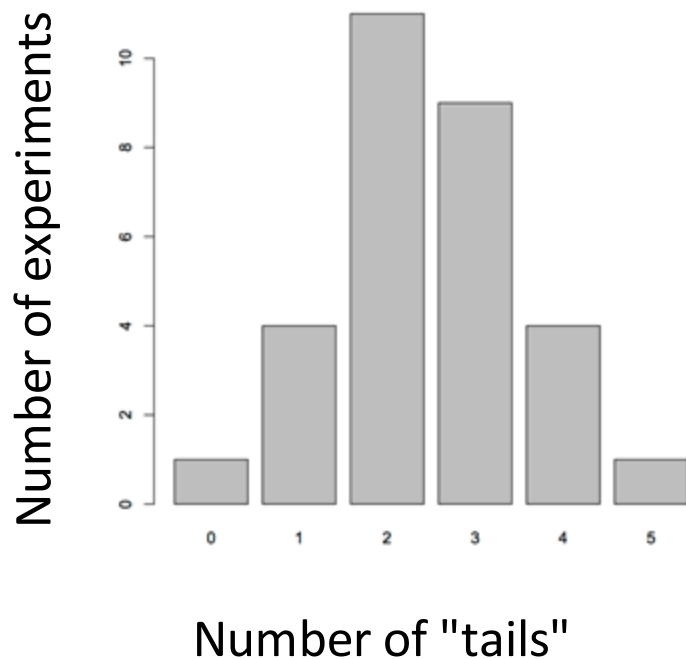


$P(\text{heads}) = 0.5$



$P(\text{tails}) = 0.5$

What is the distribution of tails  
(alternate alleles) do we expect to see  
after 5 tosses (sequence reads)?



R code:

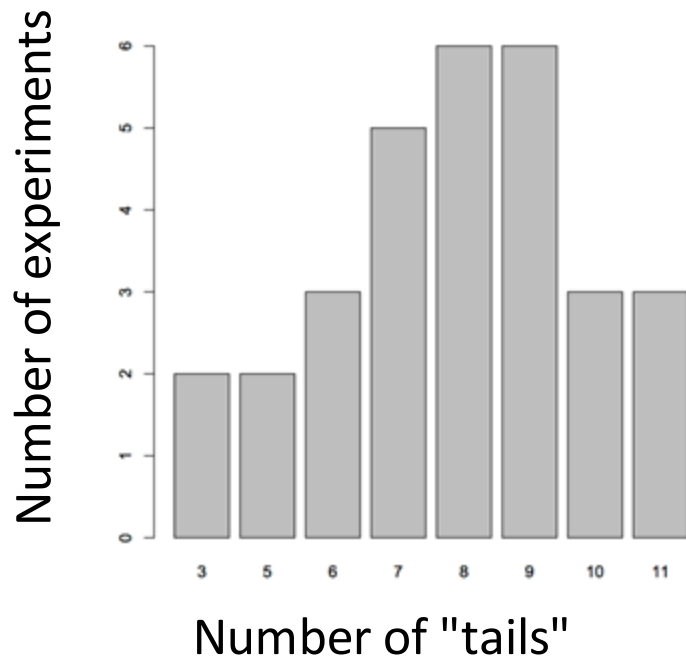
```
barplot(table(rbinom(30, 5, 0.5)))
```

30 experiments (students tossing coins)

5 tosses each

Probability of Tails

What is the distribution of tails  
(alternate alleles) do we expect to see  
after 15 tosses (sequence reads)?



R code:

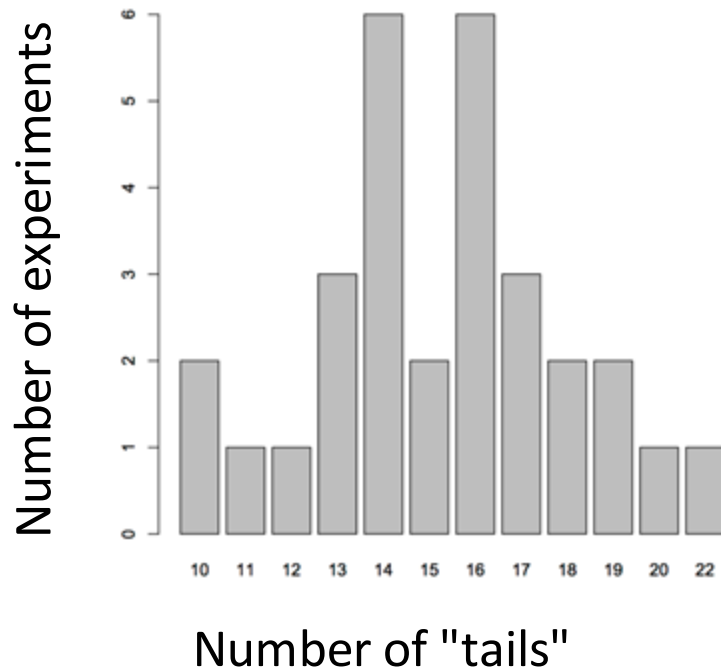
```
barplot(table(rbinom(30, 15, 0.5)))
```

30 experiments (students tossing coins)

15 tosses each

Probability of Tails

What is the distribution of tails  
(alternate alleles) do we expect to see  
after 30 tosses (sequence reads)?



R code:

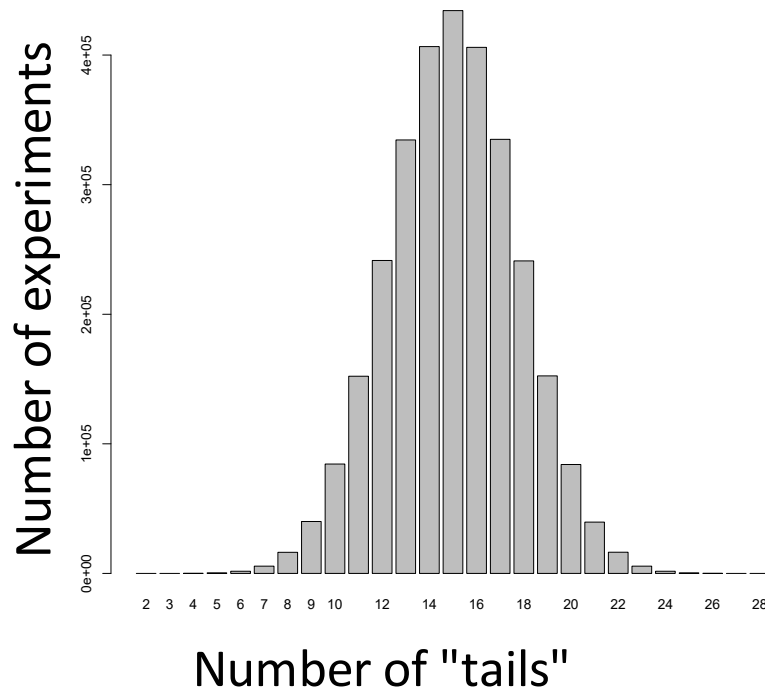
```
barplot(table(rbinom(30, 30, 0.5)))
```

30 experiments (students tossing coins)

30 tosses each

Probability of Tails

What is the distribution of tails  
(alternate alleles) do we expect to see  
after 30 tosses (sequence reads)?



R code:

```
barplot(table(rbinom(3e6, 30, 0.5)))
```

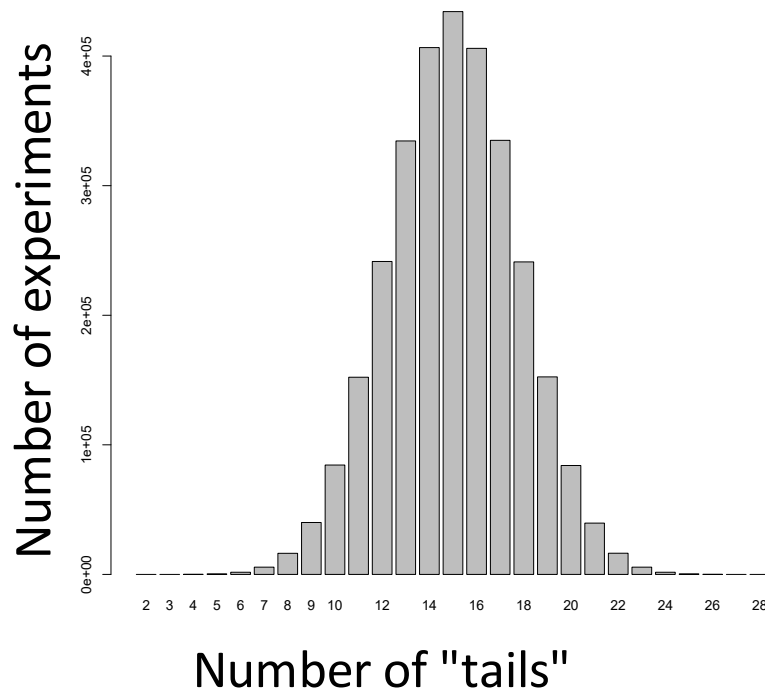
3M experiments (students tossing coins)

30 tosses each

Probability of Tails



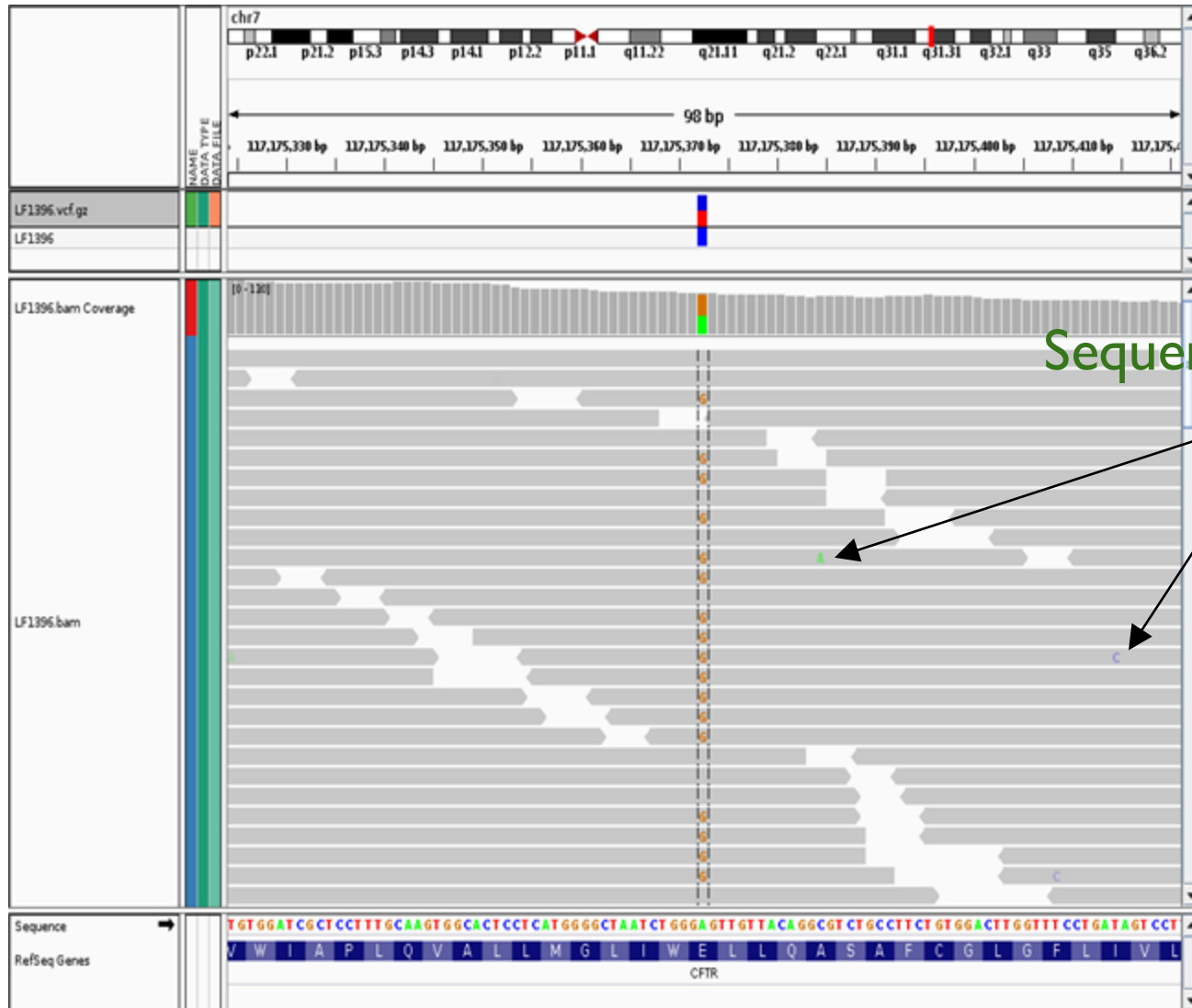
So, with 30 tosses (reads), we are much more likely to see an even mix of alternate and reference alleles at a heterozygous locus in a genome



This is why at least a "30X" (30 fold sequence coverage) genome is recommended: it confers sufficient power to distinguish heterozygous alleles and from mere sequencing errors

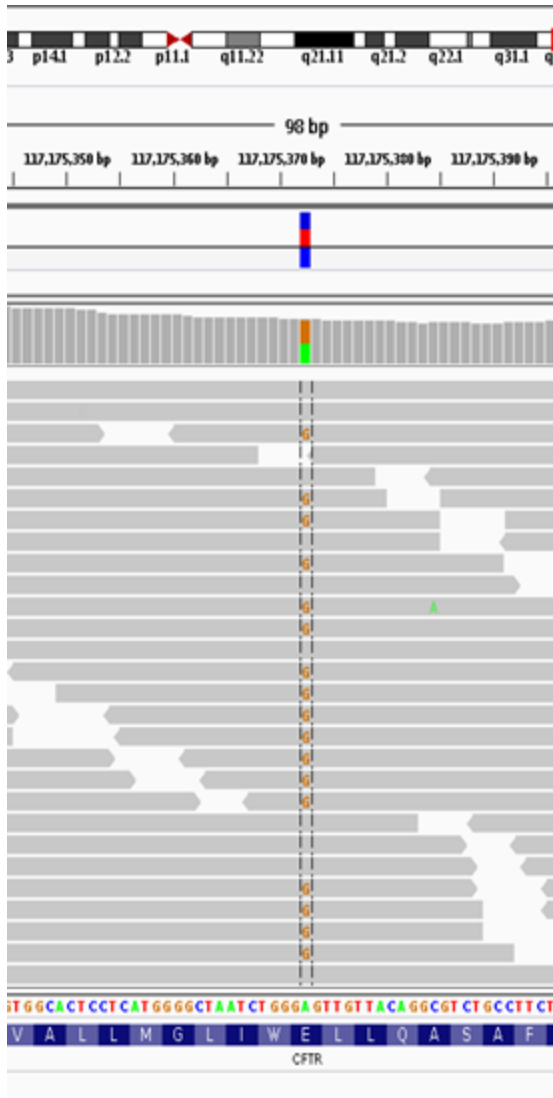
$$P(3/30 \text{ het}) <?> P(3/30 \text{ err})$$

# Sequencing errors fall out as noise (most of the time)



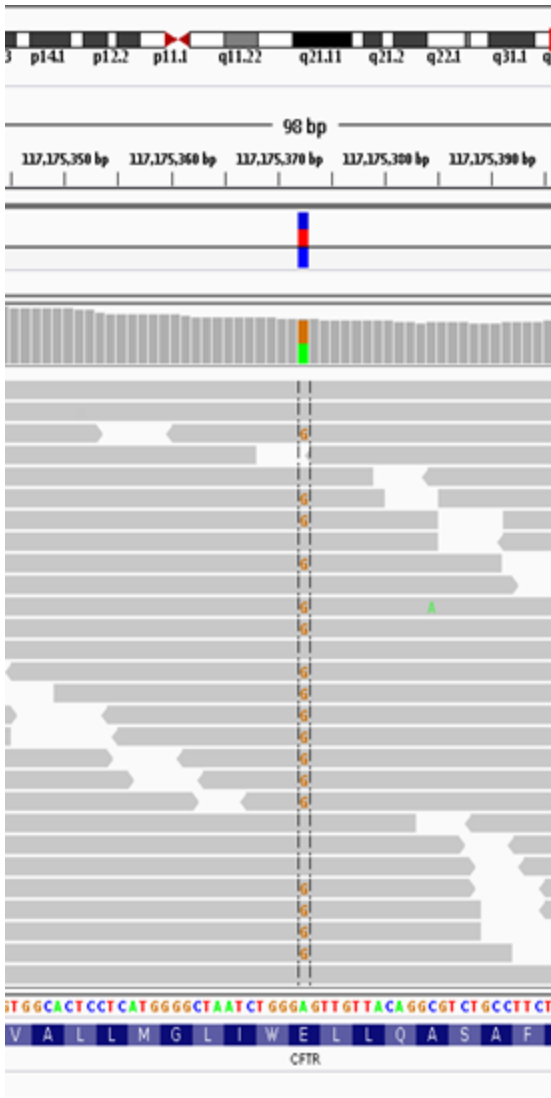
Sequencing errors

# What information is needed to decide if a variant exists?



- Depth of coverage at the locus
- Bases observed at the locus
- The base qualities of each allele
- The strand composition
- Mapping qualities
- Proper pairs?
- Expected polymorphism rate

# Bayesian SNP calling



$$P(\text{SNP} | \text{Data}) = \frac{P(\text{Data} | \text{SNP}) * P(\text{SNP})}{P(\text{Data})}$$

# PolyBayes: The first statistically rigorous variant detection tool.

letter

© 1999 Nature America Inc. • <http://genetics.nature.com>

## A general approach to single-nucleotide polymorphism discovery

Gabor T. Marth<sup>1</sup>, Ian Korf<sup>1</sup>, Mark D. Yandell<sup>1</sup>, Raymond T. Yeh<sup>1</sup>, Zhijie Gu<sup>2</sup>, Hamideh Zakeri<sup>2</sup>, Nathan O. Stitzel<sup>1</sup>, LaDeana Hillier<sup>1</sup>, Pui-Yan Kwok<sup>2</sup> & Warren R. Gish<sup>1</sup>

Bayesian  
posterior  
probability

Base call +  
Base quality

Expected (prior)  
polymorphism rate

$$P(SNP) = \sum_{\text{all variable } S} \frac{\frac{P(S_1 | R_1) \cdots P(S_N | R_N)}{P_{Prior}(S_1) \cdots P_{Prior}(S_N)} \cdot P_{Prior}(S_1, \dots, S_N)}{\sum_{S_{i_1} \in [A, C, G, T]} \cdots \sum_{S_{i_N} \in [A, C, G, T]} \frac{P(S_{i_1} | R_1)}{P_{Prior}(S_{i_1})} \cdots \frac{P(S_{i_N} | R_1)}{P_{Prior}(S_{i_N})} \cdot P_{Prior}(S_{i_1}, \dots, S_{i_N})}$$

Probability of observed base composition  
(should model sequencing error rate)

# PolyBayes: The first statistically rigorous variant detection tool.

*letter*

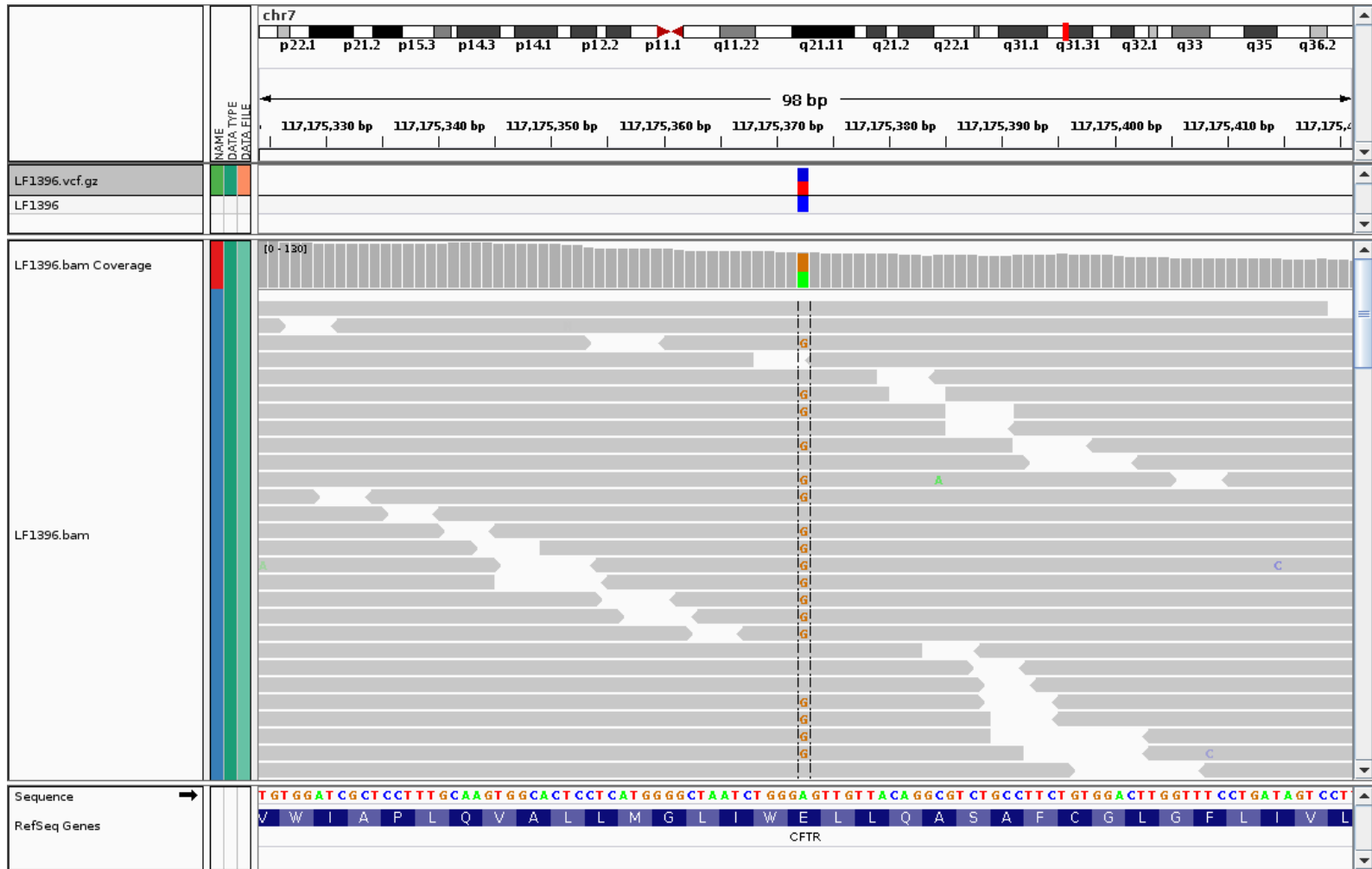
© 1999 Nature America Inc. • <http://genetics.nature.com>

## **A general approach to single-nucleotide polymorphism discovery**

Gabor T. Marth<sup>1</sup>, Ian Korf<sup>1</sup>, Mark D. Yandell<sup>1</sup>, Raymond T. Yeh<sup>1</sup>, Zhijie Gu<sup>2</sup>, Hamideh Zakeri<sup>2</sup>,  
Nathan O. Stitzel<sup>1</sup>, LaDeana Hillier<sup>1</sup>, Pui-Yan Kwok<sup>2</sup> & Warren R. Gish<sup>1</sup>

This Bayesian statistical framework has been adopted by most other modern SNP/INDEL callers such as FreeBayes, GATK, and samtools

# VCF Format



| #CHROM | POS       | ID | REF | ALT | QUAL | FILTER | INFO   | FORMAT | LF1396 |
|--------|-----------|----|-----|-----|------|--------|--------|--------|--------|
| chr7   | 117175373 | .  | A   | G   | 90   | PASS   | AF=0.5 | GT     | 0/1    |

# VCF Format

## Example

**VCF header**

```
##fileformat=VCFv4.0
##fileDate=20100707
##source=VCFtools
##reference=NCBI36
##INFO=<ID=AA,Number=1,Type=String,Description="Ancestral Allele">
##INFO=<ID=H2,Number=0,Type=Flag,Description="HapMap2 membership">
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=GQ,Number=1,Type=Integer,Description="Genotype Quality (phred score)">
##FORMAT=<ID=GL,Number=3,Type=Float,Description="Likelihoods for RR,RA,AA genotypes (R=ref,A=alt)">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##ALT=<ID=DEL,Description="Deletion">
##INFO=<ID=SVTYPE,Number=1,Type=String,Description="Type of structural variant">
##INFO=<ID=END,Number=1,Type=Integer,Description="End position of the variant">
```

**Mandatory header lines**

**Optional header lines** (meta-data about the annotations in the VCF body)

**Body**

| #CHROM | POS | ID  | REF | ALT   | QUAL | FILTER | INFO               | FORMAT   | SAMPLE1  | SAMPLE2 |
|--------|-----|-----|-----|-------|------|--------|--------------------|----------|----------|---------|
| 1      | 1   | .   | ACG | A,AT  | .    | PASS   | .                  | GT:DP    | 1/2:13   | 0/0:29  |
| 1      | 2   | rs1 | C   | T,CT  | .    | PASS   | H2;AA=T            | GT:GQ    | 0/1:100  | 2/2:70  |
| 1      | 5   | .   | A   | G     | .    | PASS   | .                  | GT:GQ    | 1/0:77   | 1/1:95  |
| 1      | 100 | .   | T   | <DEL> | .    | PASS   | SVTYPE=DEL;END=300 | GT:GQ:DP | 1/1:12:3 | 0/0:20  |

**Reference alleles** (GT=0)

**Alternate alleles** (GT>0 is an index to the ALT column)

**Deletion**

**SNP**

**Large SV**

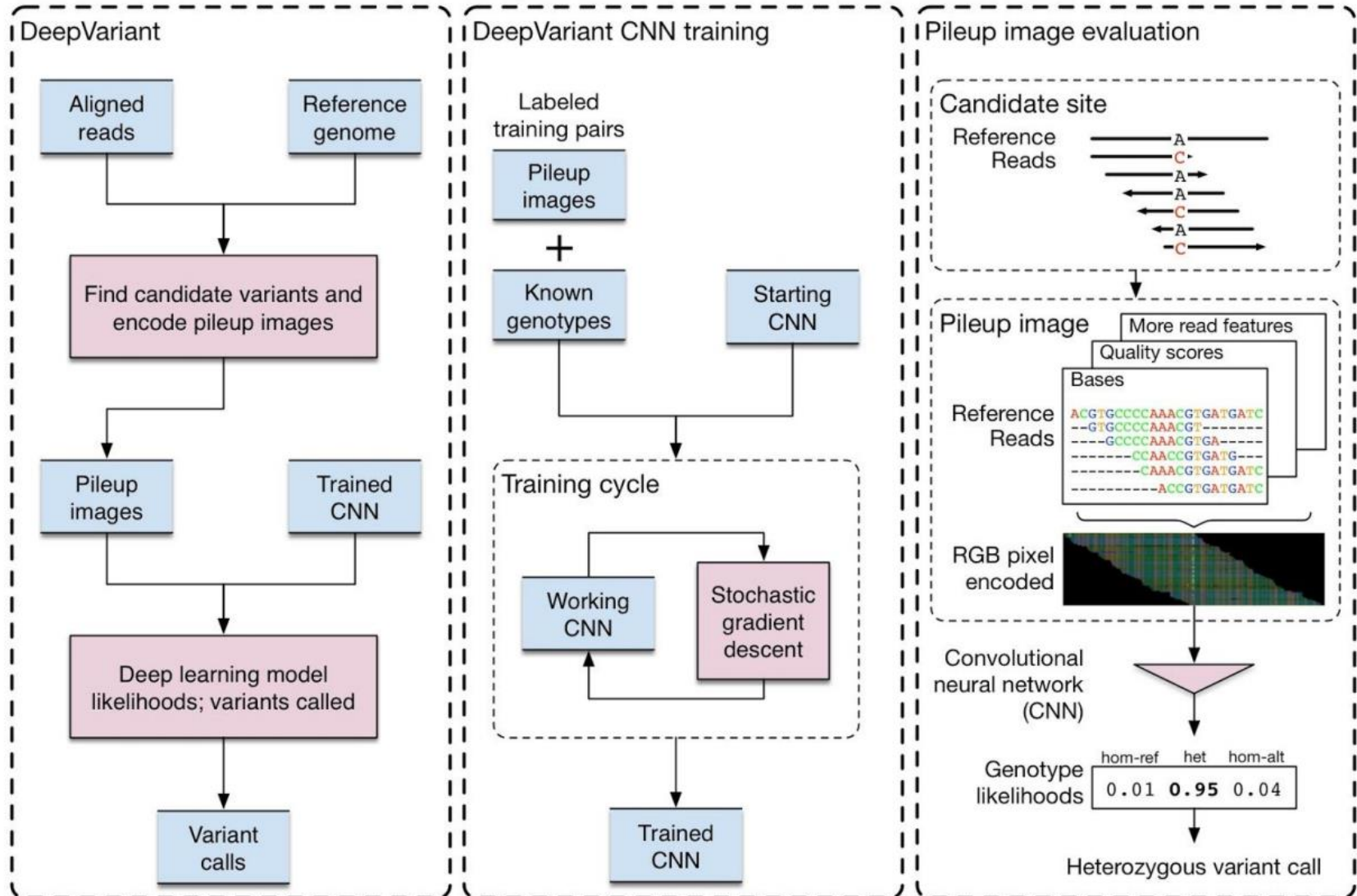
**Insertion**

**Other event**

**Phased data** (G and C above are on the same chromosome)



# DeepVariant



Creating a universal SNP and small indel variant caller with deep neural networks

Poplin et al. (2018) Nature Biotechnology. <https://www.nature.com/articles/nbt.4235>